

DISEÑO WEB 2

Actividad Individual

Verificación de contribución real y dominio individual del Proyecto Integrador

Proyectos: Plataforma de Películas · Libros · Música · Juegos

Stack: PHP · MySQL/PostgreSQL · APIs · JSON · JavaScript · Frameworks · Postman

Duración total: 150 minutos (2 h 30 min) | Modalidad: Individual y presencial en clase

1. Nota para el docente: propósito y uso

Esta actividad se aplica DOS DÍAS ANTES de la defensa grupal. Su propósito es generar evidencia individual suficiente para contrastar el nivel de dominio que cada estudiante posee sobre el proyecto integrador que desarrolló con su grupo. Los equipos están conformados por tres estudiantes y se asume que los integrantes participaron en distintas partes del desarrollo, pudiendo haber intervenido en FrontEnd, BackEnd, Base de Datos, JavaScript, APIs, JSON, Postman y otras tareas. Por esta razón, NO se asignarán retos diferentes por estudiante ni se separará artificialmente el trabajo mediante roles exclusivos

Todos los estudiantes realizarán la misma estructura de evaluación y tendrán acceso a retos equivalentes. La individualización se producirá mediante la ejecución sobre su propia copia del proyecto, las decisiones que tome durante la modificación, las evidencias generadas y su capacidad de explicar lo realizado. El resultado de esta actividad podrá utilizarse para:

- Asignar una nota individual diferenciada dentro de la valoración del Proyecto Integrador
- Detectar antes de la defensa grupal diferencias significativas de participación y dominio
- Identificar estudiantes que conocen el proyecto de manera integral y estudiantes que presentan dificultades para explicar o modificar partes que afirman haber desarrollado
- Orientar preguntas individuales durante la defensa grupal
- Generar evidencia de desempeño auténtico: comprender, modificar, ejecutar, probar y explicar

Criterio pedagógico central: no se pretende determinar quién escribió cada línea de código, sino comprobar qué grado de competencia individual posee actualmente cada integrante sobre el producto que afirma haber construido colaborativamente

2. Enunciado para el estudiante

El proyecto integrador que desarrollaste con tu grupo será defendido en 2 partes. Antes de esa defensa realizarás HOY una actividad completamente INDIVIDUAL de 150 minutos. La finalidad es demostrar que puedes comprender el funcionamiento de tu proyecto, analizar su código, realizar una modificación funcional y explicar técnicamente lo que hiciste

Reglas de la actividad

- El trabajo es estrictamente individual. No está permitido comunicarte con tus compañeros de grupo ni con estudiantes de otros grupos durante la actividad
- Debes trabajar con tu propia copia local del proyecto, funcionando en tu entorno (XAMPP, Laragon, servidor local o equivalente)
- Puedes consultar documentación oficial de los lenguajes, frameworks, librerías y Postman que hayas utilizado
- No está permitido utilizar asistentes de IA (ChatGPT, Copilot, Claude u otros) para resolver el Bloque 1 o generar la solución del reto del Bloque 2
- Todo el trabajo debe quedar registrado mediante respuestas, código modificado y capturas de evidencia
- El reto debe realizarse sobre tu proyecto real. No se permite reemplazarlo por un ejercicio genérico independiente
- Al finalizar debes poder explicar oralmente [el día de la defensa final] qué modificaste, cómo funciona y por qué elegiste esa solución

3. Estructura de la sesión (150 minutos)

Tiempo	Momento	Descripción
10 min	Apertura	Instrucciones, identificación del estudiante y autodeclaración de participación.
35 min	Bloque 1	Quiz individual: arquitectura, PHP, BD, APIs, JSON, JavaScript e integración.
10 min	Transición	Preparación de la copia local y lectura del reto común.
80 min	Bloque 2	Reto de modificación funcional sobre el propio proyecto.
15 min	Cierre	Entrega de respuestas, código, capturas y evidencias.

4. Ficha de autodeclaración de participación

Completa esta ficha antes de iniciar el quiz. No se asignará un rol exclusivo. La información servirá como referencia para contrastar tu desempeño posterior

Dato	Respuesta
Nombre completo	Pedro Aaron Espinoza Vargas
Grupo / Nombre del proyecto	IBDb_UPDS
Dominio	Películas / Libros / Música / Juegos
¿Qué partes del proyecto desarrollaste o modificaste?	Frontend / Backend / BD / JavaScript / APIs / JSON / Postman / Otras
Indica archivos, pantallas, endpoints, tablas o funcionalidades en las que participaste	Home, Destacados, Backend
¿Qué parte conoces menos o no modificaste directamente?	
Menciona una funcionalidad del proyecto que puedas explicar y modificar hoy	Consulta de API/BD

5. BLOQUE 1 – Quiz individual (35 min)

Responde individualmente y SIN consultar el código fuente. Las respuestas deben referirse a elementos reales de tu proyecto

A. Arquitectura general

1. Selección múltiple. ¿Cómo se comunican normalmente el FrontEnd y el BackEnd?

- a) El navegador accede directamente a MySQL/PostgreSQL
- b) El FrontEnd realiza peticiones HTTP al BackEnd y recibe datos, generalmente en JSON
- c) JavaScript se conecta directamente a la base de datos
- d) Postman reemplaza al FrontEnd

2. Desarrollo. Explica qué ocurre desde que el usuario hace clic en “Ver detalle de [película/libro/canción/juego]” hasta que la información aparece en pantalla. Menciona JavaScript, petición HTTP, endpoint/PHP, base de datos, JSON y HTML cuando correspondan

3. Indica qué tecnologías utilizó tu equipo y explica la función concreta que cumple cada una en TU proyecto: PHP, MySQL/PostgreSQL, JavaScript, JSON, API/Endpoints, Framework/Librería y Postman

2.- Para poder tener Detalles antes se debe haber realizado una búsqueda de un título o autor, si el título o autor no existe BACKEND realiza una petición a la API de OpenLibrary, para almacenar los datos de el libro solicitado en Base de Datos MySQL, una vez almacenado es posible Ver Detalles.
Si el libro ya existe, sera pedido desde BACKEND a MYSQL
Al Presionar Ver Detalles se obtendra el codigo de libro que deseamos visualizar renderizamos los datos de este en HTML, reemplazando el catalogo momentaneamente, los datos de los libros visualizados son cargados en cuanto se vizualiza la pagina

3.- PHP - Es el backend su funcion es comunicar el frontend con base de datos y con la API

MySQL - Guarda los datos extraidos desde API

Javascript/React/Framework/Libreria - Actua como comportamiento de la pagina, peticiones a backend por HTTP

API/Endpoints - Respuesta de tipo JSON a las peticiones de la pagina web, ya sea por medio de consulta a API o MySQL

B. JavaScript y FrontEnd

4. Analiza: `fetch('api/peliculas.php').then(response => response.json()).then(data => mostrarPeliculas(data)).catch(error => console.error(error));` Explica qué hace el código y qué papel cumple JavaScript

R.- Solicita las películas a `api/peliculas.php` una vez obtenidos los datos para el promise se convierte JSON, una vez en JSON renderiza los datos con la función `mostrarPeliculas(data)`, en caso de que exista un error en la secuencia descrita, se mostrará en consola un Error con la descripción capturada en codificación, javascript es el lenguaje de codificación en el que está escrito este código.

5. Explica qué hace: `const resultado = datos.filter(item => item.genero === 'Acción');` ¿Cómo aplicarías una operación similar en tu proyecto?

R.- Filtra los datos dependiendo del género, en este caso el género Acción; para el proyecto una función similar puede ser aplicada, pero los datos al ser paginados únicamente filtraría los libros de una sola página con 12 libros cada una, menor a la cantidad total de libros en BD

6. Explica: `document.getElementById('titulo').textContent = pelicula.titulo;` ¿qué elemento busca, qué modifica y qué información muestra?

R.- Busca el contenido del elemento con Id 'titulo', lo modifica con la información contenida por el elemento `pelicula.titulo`, y al renderizar mostrará el contenido de `pelicula.titulo`

7. Desarrollo. ¿Qué ocurre en tu proyecto si una petición fetch/AJAX falla? Explica cómo podría detectarse y cómo se podría informar al usuario

La forma de realizar peticiones fetch es por medio de búsqueda de libros no existentes en BD, en caso de fallar, la búsqueda no mostraría elementos en la página, en caso de no encontrar el contenido solicitado o fallar el fetch tiene el mismo comportamiento, se debería introducir un modal o un elemento visual que haga una distinción cuando un elemento NO es encontrado y otro que indique que fetch falló.

C. Backend / PHP

8. Menciona un archivo PHP real que hayas desarrollado o modificado. Explica qué recibe, qué procesa, qué consulta y qué devuelve.

R.- `index.php` este define el comportamiento de backend, por medio de peticiones HTTP (GET, POST, PUT, DELETE) recibe los datos y peticiones para realizar sql queries que indiquen a BD como comportarse, en caso de no existir la ruta indica error 404, la respuesta es un conjunto de datos codificados en JSON con el código HTTP describiendo el estado de la petición

9. Selección múltiple. ¿Cuál es la forma adecuada de prevenir inyección SQL?

a) Concatenar directamente el parámetro

b) Utilizar sentencias preparadas y parámetros

c) Validar solamente con JavaScript.

d) Ocultar el campo

10. Dibuja o describe dos tablas reales de tu proyecto, indicando claves primarias, claves foráneas y relación

R.-

Libro.- Almacena la información de nivel "obra" que se muestra en tarjetas, carruseles y el detalle.

autor --< libro_autor >-- libro

- libro_autor es una relación muchos a muchos (N:M) entre libro y autor.
- Un libro puede tener muchos autores; un autor puede tener muchos libros.
- La tabla libro_autor rompe la N:M creando dos relaciones de 1:N con cada tabla mediante sus clave foráneas.
- FK libro_id -> libro(id) con ON DELETE CASCADE: si se elimina un libro, se eliminan sus filas en libro_autor.
- FK autor_id -> autor(id) con ON DELETE CASCADE: si se elimina un autor, se eliminan sus filas en libro_autor. Esta relación es la que permite que la vista vw_libreria_libros genere el campo autores (los nombres concatenados que ves en las tarjetas) mediante un GROUP_CONCAT sobre esta unión.

11. Explica una consulta SQL real de tu proyecto. Explica qué devuelve y dónde se utiliza

R.-

```
SELECT * FROM vw_libreria_libros WHERE ... búsqueda/categoría ORDER BY titulo ASC  
(popularidad, rating, año) LIMIT 12 OFFSET 0
```

Devuelve los datos almacenados de los libros, es el uso principal de BACKEND

12. Menciona un endpoint real: ruta, método HTTP, parámetros, JSON devuelto y uso en el FrontEnd

Ruta: /api/libros

Método HTTP: GET

Parámetros: Buscar, categoría, orden, página, porPágina

Respuesta: { "total": 124, "rows": [{ "id": 1, "ol_work_key": "OL82563W", "titulo": "Cien años de soledad", ... }, { ... }] }

Frontend: Por medio de el archivo libraryApi.js, en la función pedir es donde se utiliza la ruta, esto es usado por Libreria.jsx para renderizar los datos.

13. Explica la diferencia entre GET y POST y proporciona un ejemplo real de tu proyecto

R.- Get solicita datos, no los modifica, POST es usado tanto para creación como para modificación de datos.

GET: GET /api/libros -> listarLibros() -> lee y devuelve el catálogo de libros en JSON

POST: POST /api/libros -> crearLibro() -> crea un libro nuevo en la BD

F. Integración

14. Describe el flujo de tu proyecto: HTML → JavaScript → PHP → Base de Datos → PHP → JSON → JavaScript → HTML. Explica qué ocurre en cada etapa

R.-

Flujo: HTML -> JavaScript -> PHP -> BD -> PHP -> JSON -> JavaScript -> HTML

HTML -> index.html carga el punto de entrada src/main.jsx y el root, donde React montará la aplicación.

JavaScript (React, frontend) -> Al renderizar Libreria.jsx, el useEffect llama a obtenerLibros(...) de libreriaApi.js, que tiene la query string y dispara fetch('/api/libros?...').

PHP (backend) -> en backend index.php recibe la petición; el router detecta el primer segmento libros con método GET y ejecuta listarLibros() , que prepara la consulta con db()->prepare(...) usando \$where, \$order y LIMIT/OFFSET.

Base de Datos -> La consulta corre contra MySQL sobre la vista vw_libreria_libros, que mediante JOIN y GROUP_CONCAT une las tablas libro, autor, libro_autor, edicion y categoria, devolviendo filas con titulo, autores concatenados, rating, etc.

PHP -> Ejecuta la consulta con \$stmt->execute(\$params) y obtiene los resultados con \$stmt->fetchAll().

JSON -> responder() hace codifica a JSON y exit, devolviendo {total, rows, fuente}.

JavaScript (frontend) -> obtenerLibros() recibe ese JSON y pasa cada fila por desdeBD(), que la normaliza, el .then() en Libreria.jsx guarda el resultado en los estados setLibros, setTotal, setFuente.

HTML -> React re-renderiza el catálogo: cada objeto de libros se convierte en (tarjeta con portada/título/autor) o (fila en lista), actualizando el DOM que el usuario ve.

15. Si al hacer clic en un botón no aparecen resultados, indica al menos tres lugares donde investigarías el problema y cómo comprobarías cada uno

R.- console.log(), Libreria.jsx, index.php, LibreriaAPI.js

si fallara algun boton el primero en visitar seria el console.log() del navegador verificaria si existe algun error visible para el elemento, Libreria.jsx ya que este contiene la mayoría de elementos visuales, que dan comportamiento a cada elemento, el siguiente seria index.php, toda la logica de conexion y obtencion de datos se concentra en este archivo principalmente.

6. BLOQUE 2 – Reto de codificación individual (80 min)

Todos los estudiantes realizarán el reto sobre su propia copia del proyecto. NO se asignarán roles ni retos distintos. El reto está diseñado para que cada estudiante pueda elegir una funcionalidad del dominio de su propio proyecto y demostrar integración entre FrontEnd, JavaScript, BackEnd/API y datos.

[index.php]

case 'libros':

if ((\$segmentos[2] ?? '') === 'portada') {

\$metodo === 'GET' ? servirPortada(idDeRuta(\$segmentos)) : responder(['error' => 'Método no permitido'], 405);

break;

}

match (true) {

\$metodo === 'GET' => listarLibros(),

\$metodo === 'POST' => crearLibro(),

\$metodo === 'PUT' => actualizarLibro(idDeRuta(\$segmentos)),

\$metodo === 'DELETE' => eliminarLibro(idDeRuta(\$segmentos)),

default => responder(['error' => 'Método no permitido'], 405),

};

break;

...

default:

responder(['error' => 'Ruta no encontrada'], 404);

}

} catch (Throwable \$e) {

responder(['error' => \$e->getMessage()], 500);

}

```

function listarLibros(): void
{
    $buscar = trim($_GET['buscar'] ?? '');
    $categoria = trim($_GET['categoria'] ?? '');
    $orden = $_GET['orden'] ?? 'default';
    $pagina = max(1, (int)($_GET['pagina'] ?? 1));
    $porPagina = min(50, max(1, (int)($_GET['porPagina'] ?? 12)));

    $mapaOrden = [
        'default' => 'id',
        'alphabetical' => 'titulo ASC',
        'popularity' => 'popularidad DESC, votos DESC',
        'rating' => 'rating DESC, votos DESC',
        'newness' => 'anio DESC',
    ];
    $ordenSql = $mapaOrden[$orden] ?? 'id';

    $where = '';
    $params = [];
    if ($buscar !== '') {
        $where = 'WHERE titulo LIKE ? OR autores LIKE ?';
        $params = ["%{$buscar}%", "%{$buscar}%"];
    }
    if ($categoria !== '') {
        $where .= ($where ? ' AND ' : 'WHERE ')
            . 'id IN (SELECT lc.libro_id FROM libro_categoria lc JOIN categoria c ON c.id = lc.categoria_id WHERE c.slug = ?)';
        $params[] = $categoria;
    }

    $stmt = db()->prepare("SELECT COUNT(*) FROM vw_libraria_libros {$where}");
    $stmt->execute($params);
    $total = (int)$stmt->fetchColumn();

    $fuente = 'mysql';

    if ($total === 0 && $buscar !== '') {
        $docs = consultarOpenLibrary($buscar);
        if ($docs) {
            guardarBusqueda($buscar, $docs);
            migrarDocs($docs);

            $stmt = db()->prepare("SELECT COUNT(*) FROM vw_libraria_libros {$where}");
            $stmt->execute($params);
            $total = (int)$stmt->fetchColumn();
            if ($total > 0) {
                $fuente = 'openlibrary';
            }
        }
    }

    $offset = ($pagina - 1) * $porPagina;
    $stmt = db()->prepare(
        "SELECT * FROM vw_libraria_libros {$where}
        ORDER BY {$ordenSql}
        LIMIT {$porPagina} OFFSET {$offset}"
    );
    $stmt->execute($params);

    responder(['total' => $total, 'rows' => $stmt->fetchAll(), 'fuente' => $fuente]);
}

```


File Edit View Help

My Workspace

Search

Invite Upgrade

Overview

GET http://127.0.0.1:8000/libros

Save Share

GET

http://127.0.0.1:8000/libros?orden=alphabetical&pagina=1&porPagina=3

Send

Docs Params Authorization Headers 7 Body Scripts Settings

Cookies

Query Params

Key	Value	Description	Bulk Edit	...
orden	alphabetical			
pagina	1			
porPagina	3			
Key	Value	Description		

Body

200 OK · 56 ms · 1.9 KB

JSON Preview Visualize

1 {

2 "total": 98,

3 "rows": [

4 {

5 "id": 7,

6 "ol_work_key": "OL262496W",

7 "titulo": "A Study in Scarlet",

8 "autores": "Arthur Conan Doyle",

9 "autor_key": "OL161167A",

10 "isbn_13": "9781986913867",

11 "portada_url": "https://covers.openlibrary.org/b/id/13405534-M.jpg",

12 "enlace_lectura": "https://archive.org/details/crime_etrange_2206_librivox",

13 "anio": 1887,

14 "rating": "3.98",

15 "votos": 121,

16 "popularidad": 521,

200 OK

Request successful. The server has responded as required.

New Chat

Describe what you need. Press @ for context, / for Skills.

Auto

Connect Git Console Terminal

0 0

Globals Vault Tools

File Edit View Help

My Workspace

Search

Invite Upgrade

Overview

GET http://127.0.0.1:8000/noexiste

Save Share

GET

http://127.0.0.1:8000/noexiste

Send

Docs Params Authorization Headers 7 Body Scripts Settings

Cookies

Query Params

Key	Value	Description	Bulk Edit	...
Key	Value	Description		

Body

404 Not Found · 21 ms · 345 B

JSON Preview Debug with AI

1 {

2 "error": "Ruta no encontrada"

3 }

Add assertions for libros pagination

Run a negative-case for noexiste

Create a parameterized pagination run

New Chat

Describe what you need. Press @ for context, / for Skills.

Auto

Connect Git Console Terminal

0 0

Globals Vault Tools

